

ClearTax: Frontend Architecture & App Flow

This document provides an extensive summary of the Next.js `client` directory, covering the overall application flow, directory structure, state management, and the core user journey within the ClearTax application.

1. Application Flow & User Journey

The frontend acts as a responsive, real-time interface for users to upload and manage bulk invoice data.

1.1 Authentication Flow (`/login` , `/signup`)

- Users land on the `LandingPage.jsx` which introduces the product.
- Authentication is handled via dedicated `/login` and `/signup` routes.
- Once authenticated, a JWT token is stored securely, and the user's session state is managed via **Zustand** (`store/auth.store.js`).
- Axios interceptors (`lib/axios.js`) automatically attach this token to all subsequent backend API requests.

1.2 The Protected Dashboard (`/app/dashboard`)

- Upon successful login, users are routed to the protected (`app`) route group.
- The `layout.js` inside (`app`) enforces authentication checks and renders the main application shell (Sidebar/Navbar).
- The `dashboard` provides a high-level overview of reconciliation analytics, utilizing **Recharts** for visual representations of Match vs. Mismatch metrics.

1.3 The Upload Workflow (`/app/upload`)

- Users navigate to the `upload` route to drag-and-drop CSV files.
- The `upload.store.js` manages the UI state (e.g., uploading progress, drag-and-drop states).
- The file is sent to the Express server using a multi-part form data request via Axios.
- The server responds with a `jobId` , which the client uses to subscribe to real-time progress updates.

1.4 Real-time Progress & Results (`/app/results` , `/app/library`)

- The client periodically polls or listens via WebSockets/SSE for job updates using the `jobId` .
- Once processed, users navigate to `results` to view a paginated, scrollable table of all processed invoices.
- **Data Fetching: React Query (@tanstack/react-query)** is heavily used here to cache data, manage loading states, and handle background refetching to ensure the UI stays synchronized with the backend DB.

2. Directory Structure Breakdown (`client/`)

The `client` directory uses the Next.js App Router paradigm, ensuring clean separation of server components, client components, and application logic.

`src/app/` (Routing & Pages)

- **(`app`)** : A Next.js Route Group containing all protected pages. Includes `dashboard` , `library` , `profile` , `reports` , `results` , and `upload` .

- **login/ & signup/** : Public authentication pages.
- **LandingPage.jsx** : The public-facing marketing and introduction page.
- **globals.css** : Global Tailwind CSS imports and base custom styles.

src/components/ (UI & Layout)

- **layout/** : Contains structural components like the **Sidebar** , **Navbar** , and **Footer** used across the **(app)** routes.
- **ui/** : Contains reusable, atomic UI components built using **Radix UI** primitives and styled with **Tailwind CSS** (e.g., Buttons, Modals, Dropdowns). Ensures UI consistency.

src/store/ (Global State Management)

Uses **Zustand** for lightweight, unopinionated global state.

- **auth.store.js** : Manages user session, JWT token, and login status.
- **jobs.store.js** : Tracks the status of background jobs (pending, processing, completed).
- **upload.store.js** : Manages temporary state related to the file upload UI.

src/hooks/ (Custom React Hooks)

- **useAuth.js** : Encapsulates authentication logic and redirects.
- **useFetch.js** : A wrapper hook for executing and handling API calls.
- **useJobs.js** , **useUpload.js** , **useInvoices.js** : Domain-specific hooks abstracting React Query data fetching logic for cleaner components.

src/lib/ (Utilities & Configuration)

- **axios.js** : The core Axios instance configured with base URLs and interceptors for attaching Authorization headers and handling 401 (Unauthorized) responses.
- **mock-data.js** : Contains fallback or development mock data for UI testing.
- **utils.js** : Helper functions (e.g., date formatting via **date-fns** , class merging using **clsx** and **tailwind-merge**).

3. Core Technologies Used (Frontend)

- **Next.js 14/15**: App Router, Server/Client components for optimized rendering.
- **React 19**: Leveraging the newest React features.
- **React Query**: For intelligent data fetching, caching, and state synchronization with the backend.
- **Zustand**: For simple global client state (Auth, UI states).
- **Tailwind CSS & Radix UI**: For rapid, accessible, and responsive styling.
- **React Hook Form & Zod**: For complex form validation (especially in login/signup flows).
- **Recharts**: For dashboard charts.
- **Framer Motion**: For micro-animations and smooth page transitions.